Requirement Engineering

⇒ Requirements engineering:

Requirements engineering is the process of deciding what services a customer needs from a system and the rules the system must follow. The requirements are descriptions of these services and rules.

⇒ Requirement:

A requirement can be simple or detailed description what a system must do. Requirements can serve two purposes: they can be used to get a contract, meaning they need to open to interpretation, or they can define the contract in detail.

interpretation.

Requirements can be understood in

in different ways by different people.

⇒Requirements abstraction:

When a company wants to hire another company to develop a large software project, it must describe its needs in a general way so that different companies can offer different solutions. After a company is chosen, more detailed requirements are written so the client understands and can approve what the software will do. Both these documents are called the requirements document for the system.

⇒Types of requirement:

. User requirements:

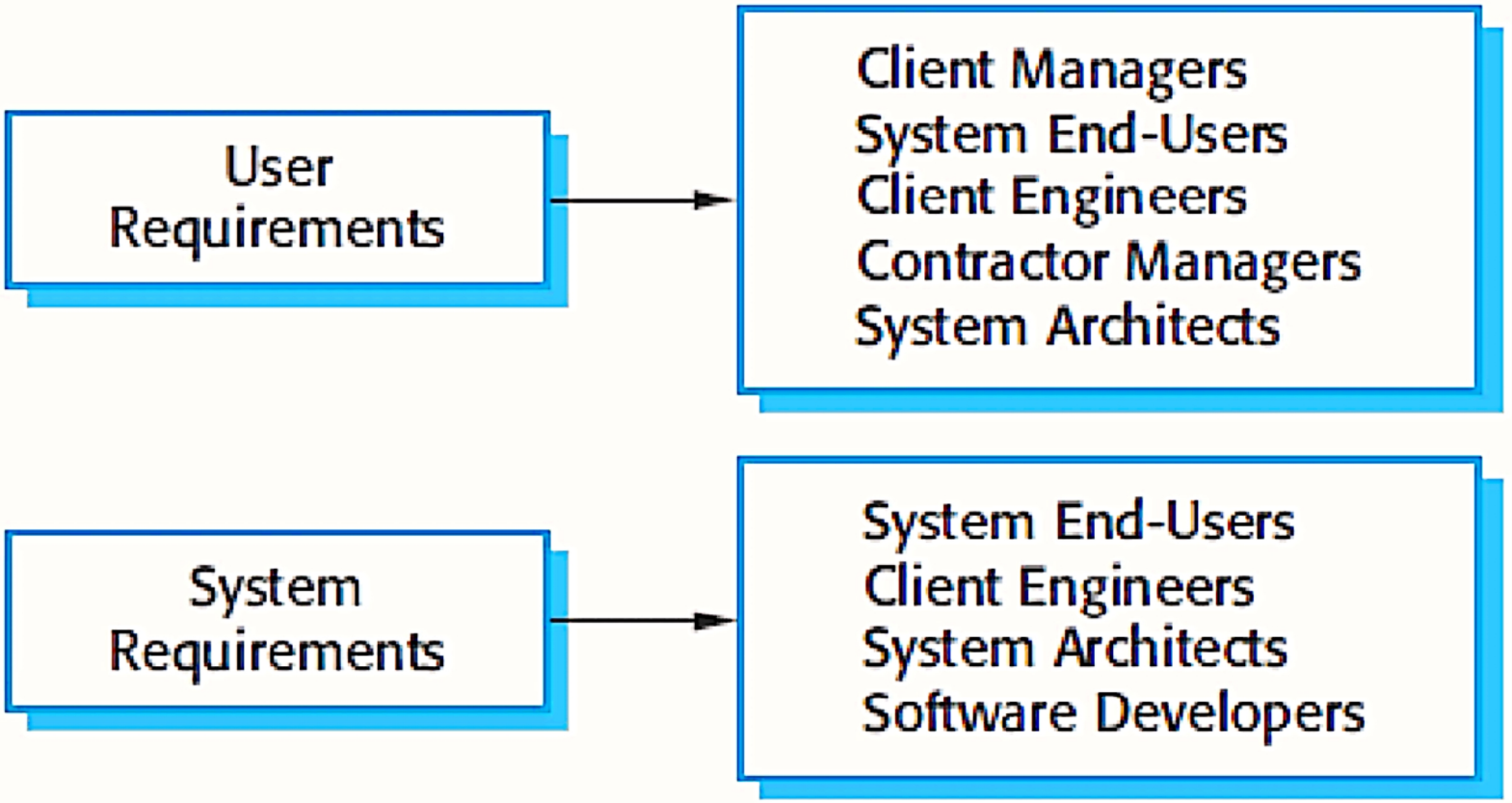
User requirements are written in natural language with diagrams and are meant for customers.

- System requirements:
System requirements are more detailed and structured, describing the system's functions, services and constraints. These are often used in contracts between clients and developers.

⇒ Functional and non-functional requirements:

- Functional requirements:
Functional requirements describe what the system should do, how it should react to specific inputs and how it should behave in different situations. For example, the system might need to let users search appointment lists, generate daily lists of patients, and identify staff members with an 8-digit number.

Readers of different types of requirements specification



⇒ Requirements imprecision:

If requirements are not clearly stated, it can cause problems. Ambiguous requirements can be interpreted in different ways by developers and users. For example, the term "search" might mean searching for a patient name across all appointments to the user, but developer might think it means searching within an individual clinic.

⇒ Requirements completeness and consistency:

Requirements should describe all needed functionality and should not have conflicting descriptions. However, it is challenging to achieve complete and consistent requirements in practice.

⇒ Non-functional requirements:

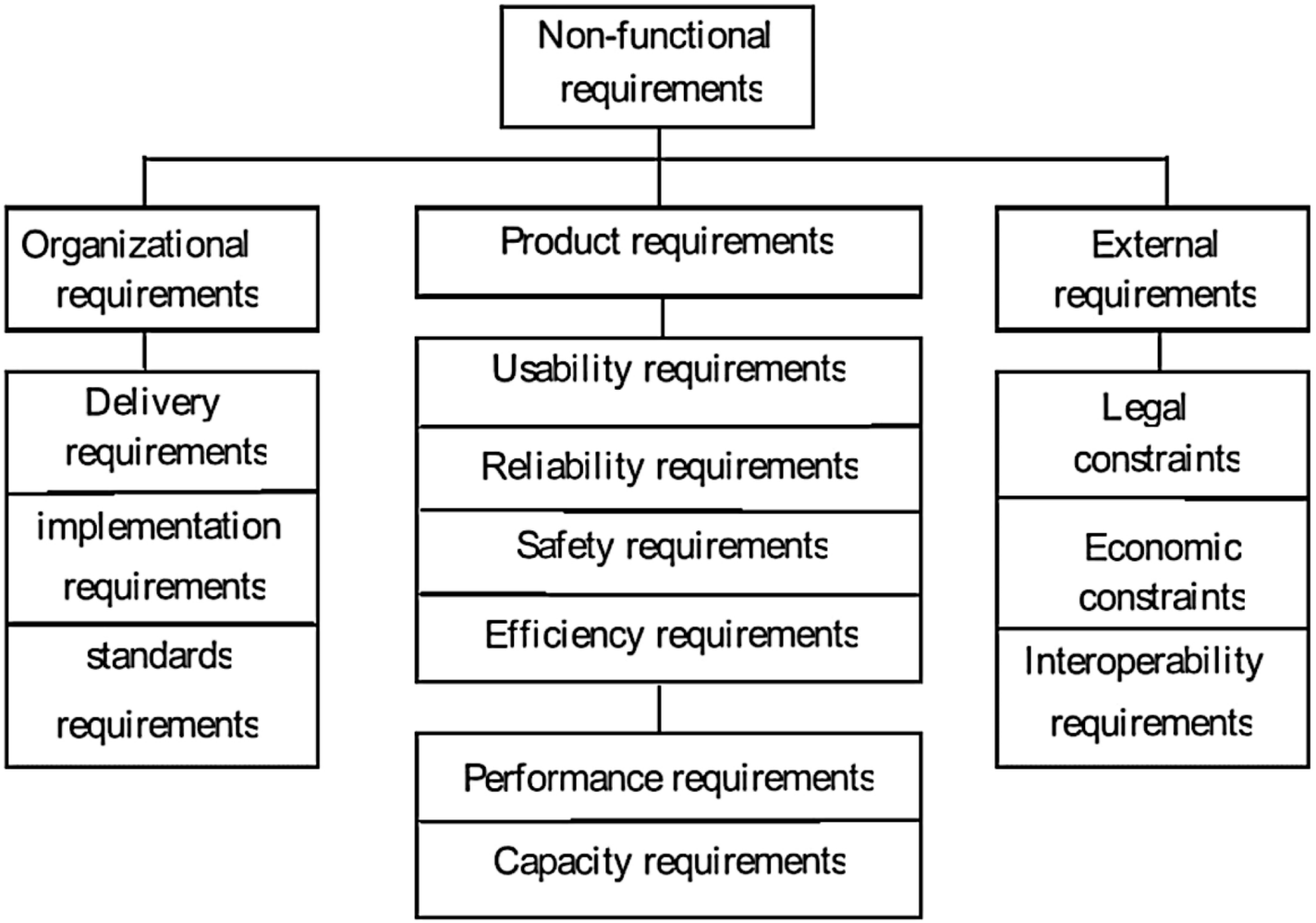
Non-functional requirements describe system properties and constraints, like reliability, response time and storage needs. These often apply to the system as a whole rather than specific features. Non-functionals requirements can be more critical than functional requirements because if they are not met, the system might be useless. For instance, if an aircraft system does not meet its reliability requirements, it will not be certified as safe for operation.

⇒ Non-functional classification:

- Product requirement:
  Specify system behaviour, like speed and reliability.

- Organisational requirement:

Types of Non-Functional Requirements



Organisational requirements come from company policies and procedures like process standards and development requirements.

- **External requirements:**
External requirements come from outside the systems and its development process, like regulatory and legislative requirements.

⇒ Non-functional requirements implementation:

Non-functional requirements can affect the overall system architecture rather than indivual components. For example, to meet performance needs, the system might need to minimize communication between parts. A single non-functional requirement, like a security requirement, might generate several related functional requirements

⇒ Usability requirements:

The system should be easy to use. For example, medical staff should be able to use all system functions after four hours of training, and after this training, the average number of errors made by experienced users should not exceed two per hour.

⇒ Domain requirements:

Domain requirements come from the system's application area rather the specific user needs. They might add new functional requirements, constrain existing ones, or specify how certain calculations must be carried out. If domain requirements are not met, the system might not work. For example, in a train control system, domain requirements might include specific calculations for train deceleration based on gradient and type of train.

⇒ Domain requirements problems:

Domain requirements can be hard to understand because they use specific language that might not be familiar to software engineers. Also, domain experts might assume some requirements are obvious and not explicitly state them, leading to implicit requirements.